SAHARA: Safety-Weighted, Privacy-Preserving On-Device Sound Awareness for Deaf and Hard-of-Hearing Users in Indian Homes

Raj Singh, Prachi Teotia, Advika [TBD: surname], Mahi [TBD: surname], and [TBD: guide name]

[TBD: Department], Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS, Mumbai, India,
[TBD: corresponding e-mail]

Abstract. Deaf and hard-of-hearing (DHH) people miss safety-critical sounds such as smoke alarms and sirens, yet sound-recognition aids rarely report how often they alert falsely, often rely on the cloud, and are seldom tested outside Western homes. We present SAHARA, a fully on-device Android system that runs a frozen YAMNet encoder with a 296k-parameter head and sends class-coded alerts to a proposed Rs. 870 ESP32 haptic wristband. Each class's threshold and temporal rule are tuned jointly against an explicit false-alarm budget, with safety classes weighted three times. SAHARA also shows open-set notices from YAMNet's 521 classes and alerts users when their name is called, naming enrolled callers. On held-out public data it reaches 0.778 mAP at 3.5 false alarms per hour of background audio, and tuning cuts false alarms from 34.1 to 6.1 per hour. On 329 recordings from Indian homes and streets, cross-validated mAP rises from 0.621 to 0.822 when recordings from the same sessions join training, but not when whole sessions are held out (0.600), which points to per-home calibration.

Keywords: assistive technology, deaf and hard of hearing, sound event detection, on-device machine learning, haptic feedback, keyword spotting

1 Introduction

More than 430 million people worldwide live with disabling hearing loss [26], and the 2011 Census of India recorded over five million people with a hearing disability [19]. For them, everyday acoustic events carry information that hearing people take for granted: a doorbell signals a visitor, a microwave beep signals that food is ready, a smoke alarm or approaching siren signals danger, and a family member calling their name signals that someone needs them. Surveys of DHH users consistently rank alarms and safety sounds as the most important to be notified about, and they prefer glanceable, wearable and private notifications [1, 3].

Deep audio embeddings trained on AudioSet [5] have made sound recognition on phones and watches practical [11–13, 15]. Three gaps remain for real-world assistive deployment. First, systems are usually evaluated with symmetric,

threshold-free metrics such as accuracy or mean average precision, yet what users experience is a stream of alerts: a missed fire alarm can be fatal, and frequent false alerts lead users to ignore or disable the aid [11]. Second, many prototypes stream audio or features to a server, which is unacceptable for an always-listening device in the home. Third, public benchmarks come mostly from Western acoustic environments; Indian homes add pressure-cooker whistles, temple bells, street hawkers, ceiling fans and monsoon rain, and their doorbells are often melodic chimes.

This paper presents SAHARA (Sound-based Alert and Haptic Assistance for Real-time Awareness), an end-to-end system that addresses these gaps. Our contributions are:

1. **Alerting tuned to a false-alarm budget.** Each class's score threshold and the number of recent windows that must exceed it are chosen jointly on validation data to maximise a safety-weighted share of events alerted, subject to a total budget of 4 false alarms per hour. On the same model this cuts false alarms on test audio from 34.1 to 6.1 per hour (Sect. 4).
2. **Privacy by construction.** All inference runs on the phone. Audio stays in a one-second ring buffer in RAM, the release build has no Internet permission, and the wristband receives only a class identifier and an intensity byte (Sect. 3).
3. **Personal and open-set awareness.** A speech-embedding spotter alerts the user when their name is called by any voice, and a speaker encoder names enrolled callers or reports "someone". YAMNet's 521 AudioSet classes provide low-priority notices for sounds outside the trained set (Sect. 5).
4. **SAHARA-IN**, 329 clips recorded in Indian homes and streets plus 107 name-call clips, with results reported separately from public data (Sect. 7).
5. **A wristband design**: a Bluetooth protocol implemented in the app, ESP32 firmware with class-coded vibration, and a bill of materials of about Rs. 870. The wristband is designed but not yet built (Sect. 6).

2 Related Work

Sound Awareness for DHH Users. Early work studied how to visualise non-speech sound [17] and how DHH users want to be notified [1, 3]. Jain et al. deployed HomeSound, an in-home sound awareness system, and found that false alerts quickly erode trust [11]; SoundWatch moved recognition to a smartwatch and analysed the trade-offs between accuracy, latency and battery life [12]. ProtoSound personalises recognition from a few user-recorded examples with prototypical networks [13, 23], and Goodman et al. studied user-driven personalisation [6]. These systems report accuracy on clips rather than alerts per hour of real audio, and they were evaluated in North American homes.

Audio Embeddings and On-Device Inference. VGGish [7], PANNs [14] and YAMNet [21], a MobileNetV1 [8] trained on AudioSet, provide transferable embeddings. Ubicoustics showed that such models transfer to real-world activity recog-

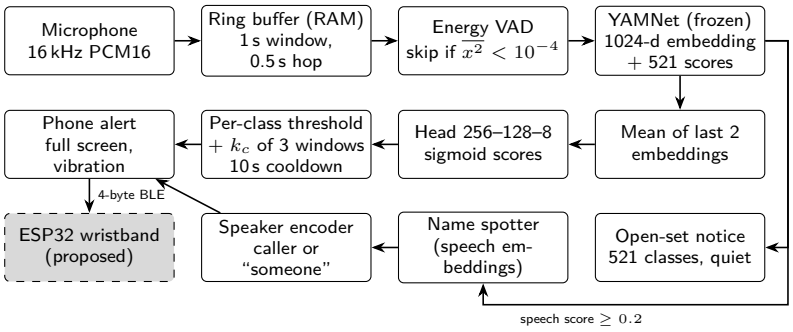


Fig. 1. SAHARA pipeline. The *white* blocks run on the phone inside an Android foreground service and the *shaded, dashed* block is the proposed wristband, designed but not yet built. Only a class identifier and an intensity byte cross the Bluetooth link; open-set notices never reach the wristband

nition with little data [15]. We use YAMNet because it is small enough for continuous inference on Android phones and has a mature TensorFlow Lite path [10].

Keyword Spotting and Speaker Recognition. Speech embeddings pretrained for keyword spotting let a small head learn a new word from few examples, including synthesised speech [16]; openWakeWord packages such embeddings for custom wake words [22]. Generalised end-to-end (GE2E) training gives compact speaker embeddings for verification [25]. We combine the two so that a name call is detected for any voice and attributed only to people the family has enrolled.

Datasets and Detection Metrics. FSD50K [4], AudioSet [5], ESC-50 [20] and DESED [24] are standard sources of domestic sound events; Mesaros et al. give a tutorial on sound event detection and its metrics [18]. Cost-sensitive learning is well established [2]; we carry the cost asymmetry past the loss into the alert rule itself.

3 System Overview

Figure 1 shows the pipeline. A native Kotlin foreground service captures 16 kHz mono audio, so detection continues with the screen off and after the app is swiped away. Samples enter a 1 s ring buffer that advances every 0.5 s. A mean-square energy gate (10^{-4}) on the newest hop skips silent audio. Each remaining window (the 0.975 s YAMNet patch) is embedded by a frozen YAMNet into a 1024-dimensional vector, and the same pass yields YAMNet's 521 AudioSet scores. The mean of the two most recent non-silent embeddings feeds a classifier head (Dense 256, ReLU, dropout; Dense 128, ReLU, dropout; Dense 8, sigmoid) that scores eight classes independently: smoke alarm, doorbell, siren, knocking,

dog bark, baby cry, glass break and appliance beep. The head has $1024 \times 256 + 256 + 256 \times 128 + 128 + 128 \times 8 + 8 = 296{,}328$ parameters (1.2 MB in 32-bit floats). The training pipeline computes exactly the same window vectors as the phone; a test asserts a maximum difference of 0.0. A Flutter interface shows a full-screen alert, keeps a history of labels, confidences and times, and offers a sensitivity slider that shifts every threshold by up to ± 2 in log-odds.

Privacy. Audio exists only as samples in the ring buffer and is overwritten every 0.5 s. Nothing is written to storage, the release manifest has no `INTERNET` permission, and neither audio, embeddings nor scores cross the Bluetooth link. Enrolment for name calls and custom sounds keeps only embedding vectors, never audio. On a Galaxy S24 we confirmed that app storage holds no audio files after listening.

4 Safety-Weighted Alerting

We call $\mathcal{S} = \{\text{smoke alarm, siren, glass break}\}$ the safety-critical classes. Each class c has a weight $w_c = 3$ for $c \in \mathcal{S}$, 1.5 for baby cry and appliance beep, and 1 otherwise. The weights enter twice: in training and in alert tuning.

4.1 Training

The head is trained on window vectors with a positive-weighted binary cross-entropy

$$\mathcal{L} = -\frac{1}{C} \sum_{c=1}^{C} \Big[w_c \, y_c \log \hat{y}_c + (1 - y_c) \log(1 - \hat{y}_c) \Big] , \tag{1}$$

so a missed positive costs up to three times as much as a false positive [2]. Sample weights balance the classes (inverse square root of frequency), give background windows eight times their balanced weight, and give the SAHARA-IN recordings ten times theirs, because they are few but closest to real use. Adam (learning rate 3×10^{-4}, batch 128) runs with early stopping on validation mAP. Among candidate runs (real data only, with synthetic mixtures, five seeds, an ensemble) we select on validation alone, by the tuned alert score defined next, subject to a precision of at least 0.70 for the safety classes.

4.2 Alert Rule

Per-window scores are noisy. Class c fires at step t when its current window is a hit ($s_{c,t} \geq \theta_c$), at least $k_c \in \{1, 2, 3\}$ of its last three windows are hits, and it has not fired within a 10 s cooldown.

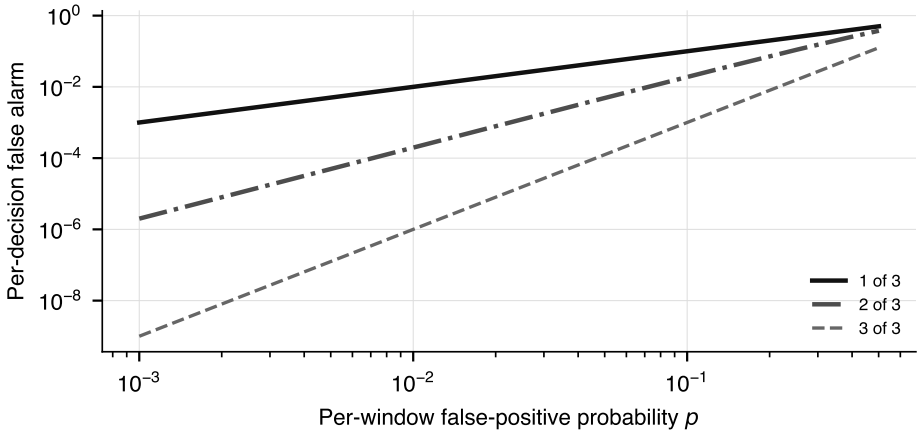


Fig. 2. Analytical per-decision false-alarm probability of the k-of-3 rule against the per-window false-positive rate p, assuming independent windows

False Alarms. If each background window were a false hit independently with probability p, the per-step false-alarm probability would be

$$F_k(p) = p \sum_{j=k-1}^{2} \binom{2}{j} p^j (1-p)^{2-j}\,, \tag{2}$$

so $F_1 = p$, $F_2 \approx 2p^2$ and $F_3 = p^3$ (Fig. 2). Overlapping windows are positively correlated, so the real reduction is smaller; this is why we measure false alarms on continuous audio instead of relying on the bound.

Latency. With window $W = 1\,\mathrm{s}$, hop $H = 0.5\,\mathrm{s}$ and $m = 2$ pooled embeddings, and assuming windows lying inside a sustained event score above threshold, the worst-case decision latency is

$$L_c \le W + mH + (k_c - 1)H + \tau\,, \tag{3}$$

where τ is the inference time per window: $2.0\,\mathrm{s} + \tau$ for $k_c = 1$ and $3.0\,\mathrm{s} + \tau$ for $k_c = 3$.

4.3 Tuning Against a False-Alarm Budget

A fixed threshold of 0.5 and a fixed k ignore how each class actually behaves in a stream. We instead replay every validation clip through the exact alert rule, one class at a time, and for every candidate (θ_c, k_c) (thresholds 0.05–0.999, $k_c \in \{1, 2, 3\}$) measure the detection rate D_c, the share of the class's clips that raise its alert, and the false-alarm rate A_c, the alerts per hour on validation audio that does not contain it, including other classes' clips. We then solve

$$\max_{\{\theta_c, k_c\}} \sum_c w_c\, D_c(\theta_c, k_c) \quad \text{subject to} \quad \sum_c A_c(\theta_c, k_c) \le B\,, \tag{4}$$

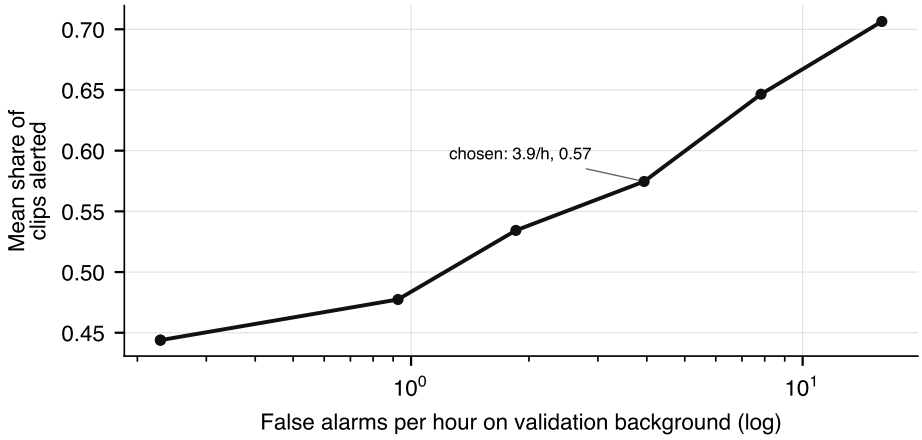


Fig. 3. Mean share of validation clips alerted against the total false-alarm budget, each point tuned separately. The leftmost point reaches zero false alarms and is drawn at 0.1

with a Lagrangian sweep. Figure 3 shows the trade-off; we chose $B = 4$ per hour, past which extra alerts buy little detection. The tuned rules differ by class: doorbell, siren, knocking, dog bark and glass break fire on one window at high thresholds (0.91–0.995), smoke alarm and baby cry, both long sounds, need two windows, and appliance beep, the class most confused with other electronic sounds, needs all three.

4.4 Class-Coded Haptics

Each alert reaches the wristband as four bytes: protocol version, command, class identifier and intensity. Safety-critical classes always use full intensity (255); for the others the intensity is the confidence scaled to $[128, 255]$. The firmware maps each class to a distinct rhythm, so the user can identify the event without looking: smoke alarm uses the "temporal-three" pattern of fire alarms [9], a siren is a rising and falling swell, a doorbell is a strong then soft "ding-dong", knocking is four quick taps, and a name call is two medium and two short buzzes (Fig. 4). The phone plays the same patterns with amplitude-controlled vibration, so a sound feels the same on either device, and a test fails the build if the two copies differ. A new alert interrupts the current one.

5 Name Calls and Open-Set Notices

Why Not More Trained Classes. The eight classes are the sounds a DHH user must not miss, and each has curated data, its own tuned rule and a place in the false-alarm budget. Every added class needs data and adds false alarms that erode trust in the others. SAHARA therefore keeps the trained set fixed and adds two layers that cannot delay or suppress a safety alert.

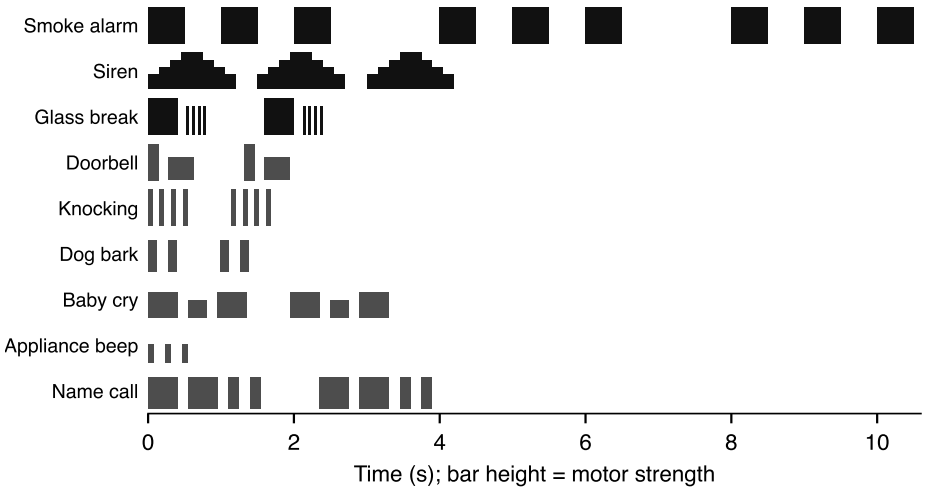


Fig. 4. Wristband vibration patterns, drawn from the firmware's timing tables on one time scale. The three safety-critical classes (top) use full strength and repeat three times or carry a distinctive onset

Open-Set Notices. YAMNet's 521 AudioSet scores are already computed in each pass. When the top class that is neither covered by the head (doorbell, siren, knock, bark, baby cry and so on) nor uninformative (speech, silence, music, television) scores at least 0.35 in two of three windows, the app posts a quiet notice such as "Water tap" or "Cat", with a 30 s cooldown per sound. Notices do not vibrate, do not reach the wristband and are suppressed while a trained class fires. They inform; they make no safety promise.

Name Calls for Any Voice. When YAMNet's speech score is at least 0.2, a second model checks whether the user's name was said. Pretrained speech embeddings [16, 22] turn the last 2 s into one 96-dimensional vector every 80 ms, and a standardised logistic regression over eight consecutive vectors scores the name. We first tried MFCC features with dynamic time warping, which performed at chance; embeddings pretrained on many speakers are what let the spotter generalise to voices it never heard. The head is trained on the team's 38 recordings of the name "Aryan", each also mixed with Indian background at 5–20 dB, against 19 recordings of similar names (Arya, Aryanshi and others) and the SAHARA-IN background scenes.

Who Is Calling. When the spotter fires, a GE2E speaker encoder [25] (a three-layer LSTM, pretrained weights, reimplemented in Kotlin) embeds the same audio into 256 numbers and compares it by cosine similarity with the mean embedding of each enrolled caller, recorded once by tapping a button while that person says the name. At or above 0.70 the alert reads "Raj is calling you"; below it, or with nobody enrolled, "Someone is calling you". Naming the wrong

Table 1. Wristband bill of materials (approximate Indian retail prices, 2026)

Component	Price (Rs.)
ESP32 DevKit V1 (WROOM-32)	350
3.7 V LiPo cell, 500 mAh	180
TP4056 USB-C charger with protection	35
3.3 V low-dropout regulator	20
10 mm coin vibration motor	40
AO3400 MOSFET, diodes, resistors, LED, switch	31
Perfboard, wiring, headers	60
Strap and enclosure	150
Total	≈870

person is worse than "someone", because with "someone" the user still learns that they are being called.

Custom Sounds. Users can also enrol a household sound from five to ten three-second recordings; its prototype is the mean of centred, normalised YAMNet window vectors, matched by cosine similarity. Our evaluation found that it separates distinctive household sounds from background but cannot distinguish names, because YAMNet describes kinds of sound rather than words, which motivated the separate name-call model.

6 Proposed Wristband

The wristband is designed but has not yet been built or measured; everything in this section is the design. Its firmware compiles for the ESP32 and the phone app already implements the alert protocol, so building it requires no change to the software. The design is an ESP32-WROOM-32 Bluetooth Low Energy peripheral with a 10 mm coin vibration motor driven through a logic-level MOSFET with PWM, a 500 mAh LiPo cell, a protected TP4056 charger and a low-dropout 3.3 V regulator. Its bill of materials totals about Rs. 870 at 2026 Indian retail prices (Table 1). To save power, the firmware clocks the CPU at 80 MHz, transmits at +3 dBm and requests a 30–50 ms connection interval with slave latency 2, which targets an alert delivery delay below about 150 ms. Below 20 % charge it lowers motor strength and transmit power, and at 3.4 V it enters deep sleep. When the phone disconnects, two long buzzes tell the user that alerts have stopped, so a silent wristband never looks the same as a quiet room. From datasheet currents we estimate 30–50 mA while connected on the development board, or about 10–15 h on the 500 mAh cell; this estimate, the alert delay and the users' ability to tell the nine patterns apart must be measured on the built device.

7 Data and Evaluation Protocol

7.1 Public Data

Target clips come from FSD50K [4], AudioSet [5] and ESC-50 [20]; background comes from 10,000 FSD50K clips without any target-like label, the 44 non-target ESC-50 categories and DESED real soundscapes [24] (Table 2). We verified every label identifier against the official vocabularies. This removed serious errors in an earlier build: four AudioSet identifiers pointed to the wrong classes ("glass break" was *Whoop*), 64 of 188 AudioSet clips did not carry their class's label, and FSD50K's generic *Alarm*, mostly telephones and car alarms, had been mapped to appliance beep. Every class now has at least 150 real clips and at least 20 in the test split.

Splits are 70/15/15, grouped by upstream recording: AudioSet and DESED clips share YouTube identifiers, and FSD50K and ESC-50 share Freesound identifiers, so the same recording never sits in training in one dataset and in test in another.

7.2 Synthetic Mixtures

Each training target clip is mixed onto three random training-split background beds at an SNR drawn uniformly from $[-5, 10]$ dB, giving 6,834 training mixtures. A separate set of 564 test mixtures uses only test clips and test beds. Synthetic results are always reported separately from real ones.

7.3 SAHARA-IN Field Recordings

Four team members recorded 329 clips on their own phones in Mumbai homes and streets, in three uploads: 269 target clips (67 knocking, 59 appliance beep, 58 doorbell, 20 each of smoke alarm, siren and glass break, 14 baby cry, 11 dog bark) and 60 background clips (street hawkers, a temple, a kitchen, television, monsoon rain, and hard negatives such as drawers, doors and a washing machine spinning without beeping). Because every recording was made under different conditions, we split by label rather than by recorder: about 40 % of each label, spread across its recording conditions, is held out for test, and each upload is split on its own so that adding clips never moves an earlier one. With so few clips, we also report five-fold cross-validation over all 329, in which every clip is scored by a model that never heard it. A further 107 clips (38 of the name "Aryan", 69 of similar names) serve the name-call model, which is evaluated leave-one-recorder-out.

7.4 Metrics

We report per-class average precision (AP) over 0.5 s windows and its mean (mAP), and, closer to what a user experiences, the share of test clips that raise the correct alert under the deployed rule and the false alarms per hour when background audio is played as one continuous stream. Model selection and alert tuning use only the validation split; the test split was used once.

Table 2. Real clips per class and source, from the committed manifests; † marks the safety-critical classes. Background also includes 605 DESED clips

Class	AudioSet	ESC-50	FSD50K	SAHARA-IN	Total (test)
Smoke alarm†	183	–	–	20	203 (35)
Doorbell	92	–	144	58	294 (57)
Siren†	34	40	132	20	226 (42)
Knocking	97	40	373	67	577 (100)
Dog bark	10	40	536	11	597 (94)
Baby cry	106	40	–	14	160 (30)
Glass break†	97	40	510	20	667 (105)
Appliance beep	261	40	206	59	566 (101)
Background	–	1,760	10,000	60	12,425 (1,876)

Table 3. Selected model on each test set

Test set	mAP	False alarms/h
Public held-out split (8 classes)	0.778	3.5 (3.4 h)
Synthetic mixtures, −5 to 10 dB (8 classes)	0.666	–
SAHARA-IN held-out split (8 classes)	0.831	see text
SAHARA-IN 5-fold CV, trained without / with it	0.621 / 0.822	–
same, whole recording sessions held out	0.621 / 0.600	–

8 Results

8.1 Detection on Held-Out Data

Table 3 summarises the selected model; Table 4 and Fig. 5 give the public test set per class. The model reaches 0.778 mAP and, played as a stream through the deployed rule, raises 3.5 false alarms per hour on 3.4 hours of public test background (mostly siren). The safety classes transfer well: 22 of 27 smoke-alarm and 28 of 34 siren clips raise an alert. Glass break has high AP (0.827) but alerts on 56 of 97 clips, because the budget forced a threshold of 0.93. Knocking (0.604) and appliance beep (0.567) are the weakest; their brief, varied sounds limit the model rather than the rule (Fig. 3). On synthetic mixtures at −5 to 10 dB, mAP falls to 0.666 and clip detection drops sharply for doorbell (to 7 of 57), knocking (3 of 100) and appliance beep (9 of 101), showing that loud background is the main remaining challenge.

8.2 Indian Conditions

In five-fold cross-validation over all 329 SAHARA-IN clips, adding them to training (weighted ten times) raises mAP from 0.621 to 0.822 and the clips that raise an alert from 45 to 100 of 269. The gains are largest where Indian sources differ most from public data: smoke alarm AP rises from 0.385 to 0.801, knocking

Table 4. Public test set per class: window AP, and test clips that raise the correct alert under the deployed rule (θ_c, k_c of 3)

Class	θ_c	k_c	AP	Clips alerted
Smoke alarm†	0.81	2	0.916	22/27
Doorbell	0.995	1	0.711	13/35
Siren†	0.91	1	0.875	28/34
Knocking	0.95	1	0.604	21/73
Dog bark	0.98	1	0.908	67/90
Baby cry	0.99	2	0.817	17/25
Glass break†	0.93	1	0.827	56/97
Appliance beep	0.90	3	0.567	17/78
Mean			0.778	0.56

from 0.482 to 0.763, doorbell, whose Indian chimes are melodic, from 0.521 to 0.824, and glass break from 0.208 to 0.476. The team's background recordings produced 1 false alarm in 5.9 minutes without the team data and 3 with it; on the held-out split the one false alarm was "siren", triggered by street sound. This is too little audio for a reliable hourly rate, but it marks Indian street sound as the main false-alarm risk.

These folds, however, deal clips at random, so other clips from the same sitting (the same phone, room and often the same doorbell or appliance) are in training. Holding out whole recording sessions instead, inferred from the recorder or, for unattributed uploads, from the phone's numbered file-name series, the team data no longer raises mAP (0.621 without, 0.600 with). It still helps doorbells (AP 0.521 to 0.691) and catches more baby cries (2 to 5 of 14) and beeps (9 to 12 of 59) in unseen homes, but knocking and glass break, each dominated by one session, get worse. Field recordings therefore teach the model a particular home's sounds far better than they teach it Indian sounds in general, and the held-out split below, which shares sessions with training, overstates transfer to a new home.

On the held-out split the selected model reaches 0.831 mAP and alerts on 37 of 105 Indian clips; given the shared sessions, we read this as performance in homes the model has heard. The previous model, trained before the third upload added 161 clips, reaches 0.698 on the same split and alerts on 31, with 2 rather than 6 of 27 knocks and 0 rather than 4 of 8 glass breaks, at the same 3.5 false alarms per hour on public background. Validation holds only public audio and cannot see this gain (the previous model scores 0.643 there against 0.634), so we select among runs trained on the current data. Knocking still transfers poorly (6 of 27 alerted although its AP is 0.812): its threshold, tuned on public validation audio, sits above most Indian knocks. Tuning thresholds on Indian validation clips, tried before the third upload, failed because six Indian knocks were too few to set a threshold; with 67 knocks now recorded, this is the next step.

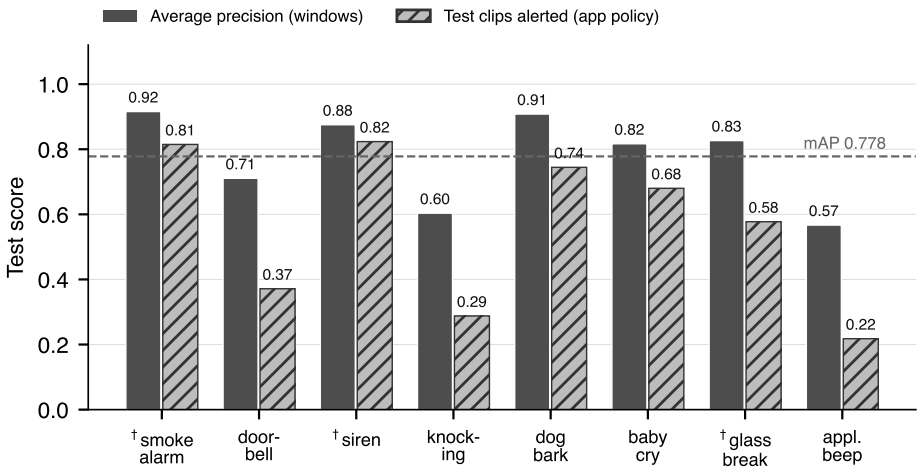


Fig. 5. Per-class test AP and share of test clips alerted under the deployed rule (public test set); † marks safety-critical classes. The gap between the bars is the price of the 4-per-hour false-alarm budget

8.3 Home Calibration and Sensitivity

Because field recordings help most in the home they come from, SAHARA lets the user calibrate: they record a sound three times where it happens (their own doorbell, door or microwave), and for each recording the app computes the highest threshold at which the deployed rule would still alert. The class's threshold is set 0.5 log-odds below the weakest recording, but never below its threshold in the most permissive tuned setting (16 false alarms per hour) and never above the current one, so calibration only moves a class within a range whose false-alarm cost was measured; an optional recording of ordinary home sound raises it again if that sound would fire. Only the threshold is stored. We measured this on the session-held-out models: three clips of an unseen session serve as the calibration recordings and the rest as the events to catch, over 20 random draws. Across 17 session–class pairs, the share of events alerted rises from 19 % to 35 %, for about one extra false alarm per hour per calibrated class on the public test background. Doorbells gain most (for example 0 to 42 % and 20 to 83 %), followed by appliance beeps and baby cries; knocking, glass break and smoke alarm do not gain, because in those sessions the model's scores stay below even the most permissive tested threshold.

The same tuning, run at budgets of 1, 2, 4, 8 and 16 false alarms per hour, gives the app's five-step sensitivity setting, so every step has a measured trade-off: on all real test audio, from 47 % of clips alerted at 2.5 false alarms per hour to 67 % at 20.6.

Table 5. Same model, three alert rules, on all real test audio

Alert rule	Clips alerted	Safety clips	False alarms/h
$\theta = 0.5$, 1 window	0.766	0.838	49.1
$\theta = 0.5$, 1 (safety) or 2 of 3 windows	0.688	0.838	34.1
Tuned (θ_c, k_c), 4/h budget (SAHARA)	0.538	0.692	6.1

Table 6. Name-call detection, leave one recorder out (38 calls of "Aryan")

Negatives	ROC AUC	Equal error rate
Background scenes (TV, kitchen, street, temple)	0.872	19 %
Similar names (Arya, Aryanshi, others; 69 clips)	0.743	32 %
Both	0.821	24 %

8.4 What the Alert Tuning Buys

Table 5 replays all 2,435 real test clips (4.6 h) through three rules on the same model. False alarms here count alerts for a class on any audio without it, including other classes' clips, so they are higher than the background-only figure above. The fixed design of a 0.5 threshold with 2-of-3 smoothing (1 window for safety classes) produces 34.1 false alarms per hour, more than one every two minutes, which no user would tolerate. Tuning brings this to 6.1 per hour while still alerting on 54 % of clips and 69 % of safety-class clips. The tuning makes this trade-off explicit and puts it under the designer's control instead of hiding it inside a threshold-free metric.

8.5 Name Calls and Callers

Table 6 reports the name-call model, leave one recorder out: the spotter is trained without any of the held-out person's clips or background scenes, then scored on them, so every tested voice is new. Similar names are the hardest case. For caller identity, leaving one clip out while the rest enrol each caller names the right person in 29 of 38 calls; with one caller left out entirely, 82 % of their calls are reported as "someone" at the 0.70 threshold. Each team member recorded on their own phone and in their own room, so these caller figures partly measure phone and room identity and are likely optimistic.

8.6 On-Device Cost

Table 7 reports the deployment on a Samsung Galaxy S24 (Android 16). The app path was verified against the evaluation pipeline: 54 test clips replayed through the app's exact code raise the same alerts in 54 of 54 cases. Laptop-speaker playback triggered smoke alarm and siren alerts with the screen on and off, but small speakers barely reproduce knocks, barks and doorbells, so those classes still need real-source tests.

Table 7. On-device cost (Samsung Galaxy S24, Android 16)

Measure	Value
Head parameters	296,328
Model files, YAMNet / head / name-call (MB)	15.0 / 1.2 / 8.1
INT8 head size, test mAP change	0.29 MB, −0.015
Release APK, arm64 (MB)	45.8
Inference per window, screen on / off (ms)	20–29 / 47–49 (hop 500)
Memory, debug build (MB PSS)	128.9
Phone battery drain while listening (%/h)	**TBD**

9 Discussion and Limitations

Detection Versus False Alarms. At 4 false alarms per hour the model alerts on about 58 % of validation events; Fig. 3 shows that relaxing the budget fourfold, to 16 per hour, raises this only to 71 %. Better detection of brief sounds such as knocks and beeps therefore needs a stronger model or more data, not a looser rule.

Small Field Set. SAHARA-IN has 329 clips, with only 20 per safety class and 8 of each in the test split. Most of it comes from a few sessions, and holding sessions out removes the gain seen with random folds, so recordings from many more households are needed before transfer to unseen Indian homes can be claimed.

Name Calls. The spotter is trained for one name and confuses similar names about a third of the time. Adding 1,781 synthetic clips of the name and of near-miss names from 27 text-to-speech voices, including Indian English and Hindi, lowered the AUC against similar names from 0.719 to 0.615 on the first upload. Fifty real near-miss recordings from the second upload did help: false calls on similar names fell from 61 % to 36 % and calls caught rose from 68 % to 79 %, and on the 19 earlier near-miss clips, which share no session with the new ones, AUC rose from 0.719 to 0.755. Caller identification must be re-measured once a family enrols on the user's own phone. The speech-embedding models are licensed for non-commercial use only.

No User Study Yet. We have not yet evaluated SAHARA with DHH participants, measured battery drain over a day, or built the wristband; its alert delay, current draw and battery life are therefore not reported. A field deployment with DHH users in Mumbai, measuring missed events, alert fatigue and recognition of the haptic patterns, is planned [3, 11].

10 Conclusion

SAHARA shows that an assistive sound-awareness system can run entirely on a phone, retain no audio, and be tuned for the number that decides whether

users keep it switched on: false alarms per hour. Tuning each class's threshold and temporal rule against an explicit budget cut false alarms more than five-fold on the same model, and recordings from Indian homes taught the model those homes' sounds, though not yet sounds in homes it has never heard; per-home calibration is the step this points to. A separate speech path lets users know when they are called by name, and by whom. We release the code, the alert-tuning method, the Bluetooth protocol, the firmware and the wristband design (about Rs. 870), whose construction and evaluation are our next step, and the SAHARA-IN protocol.

Acknowledgements. **[TBD: thank the guide and department; mention any funding]**

References

1. Bragg, D., Huynh, N., Ladner, R.E.: A personalizable mobile sound detector app design for deaf and hard-of-hearing users. In: Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS). ACM, New York (2016)
2. Elkan, C.: The foundations of cost-sensitive learning. In: Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI), pp. 973–978 (2001)
3. Findlater, L., Chinh, B., Jain, D., Froehlich, J., Kushalnagar, R., Lin, A.C.: Deaf and hard-of-hearing individuals' preferences for wearable and mobile sound awareness technologies. In: Proceedings of the CHI Conference on Human Factors in Computing Systems. ACM, New York (2019)
4. Fonseca, E., Favory, X., Pons, J., Font, F., Serra, X.: FSD50K: an open dataset of human-labeled sound events. IEEE/ACM Trans. Audio Speech Lang. Process. 30, 829–852 (2022)
5. Gemmeke, J.F., et al.: Audio Set: an ontology and human-labeled dataset for audio events. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 776–780 (2017)
6. Goodman, S.M., et al.: Toward user-driven sound recognizer personalization with people who are d/Deaf or hard of hearing. Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. 5(2) (2021)
7. Hershey, S., et al.: CNN architectures for large-scale audio classification. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 131–135 (2017)
8. Howard, A.G., et al.: MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861 (2017)
9. International Organization for Standardization: ISO 8201:2017 Alarm systems – Audible emergency evacuation signal – Requirements. Geneva (2017)
10. Jacob, B., et al.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713 (2018)
11. Jain, D., et al.: HomeSound: an iterative field deployment of an in-home sound awareness system for deaf or hard of hearing users. In: Proceedings of the CHI Conference on Human Factors in Computing Systems. ACM, New York (2020)

12. Jain, D., Ngo, H., Patel, P., Goodman, S., Findlater, L., Froehlich, J.: SoundWatch: exploring smartwatch-based deep learning approaches to support sound awareness for deaf and hard of hearing users. In: Proceedings of the 22nd International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS). ACM, New York (2020)
13. Jain, D., et al.: ProtoSound: a personalized and scalable sound recognition system for deaf and hard-of-hearing users. In: Proceedings of the CHI Conference on Human Factors in Computing Systems. ACM, New York (2022)
14. Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W., Plumbley, M.D.: PANNs: large-scale pretrained audio neural networks for audio pattern recognition. IEEE/ACM Trans. Audio Speech Lang. Process. 28, 2880–2894 (2020)
15. Laput, G., Ahuja, K., Goel, M., Harrison, C.: Ubicoustics: plug-and-play acoustic activity recognition. In: Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST). ACM, New York (2018)
16. Lin, J., Kilgour, K., Roblek, D., Sharifi, M.: Training keyword spotters with limited and synthesized speech data. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 7474–7478 (2020)
17. Matthews, T., Fong, J., Ho-Ching, F.W.-L., Mankoff, J.: Evaluating non-speech sound visualizations for the deaf. Behav. Inf. Technol. 25(4), 333–351 (2006)
18. Mesaros, A., Heittola, T., Virtanen, T., Plumbley, M.D.: Sound event detection: a tutorial. IEEE Signal Process. Mag. 38(5), 67–83 (2021)
19. Office of the Registrar General and Census Commissioner, India: Census of India 2011: Data on Disability. New Delhi (2011)
20. Piczak, K.J.: ESC: dataset for environmental sound classification. In: Proceedings of the 23rd ACM International Conference on Multimedia, pp. 1015–1018 (2015)
21. Plakal, M., Ellis, D.: YAMNet. TensorFlow Model Garden (2020). https://github.com/tensorflow/models/tree/master/research/audioset/yamnet
22. Scripka, D.: openWakeWord. GitHub (2023). https://github.com/dscripka/openWakeWord
23. Snell, J., Swersky, K., Zemel, R.: Prototypical networks for few-shot learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30 (2017)
24. Turpault, N., Serizel, R., Parag Shah, A., Salamon, J.: Sound event detection in domestic environments with weakly labeled data and soundscape synthesis. In: Proceedings of the Detection and Classification of Acoustic Scenes and Events Workshop (DCASE) (2019)
25. Wan, L., Wang, Q., Papir, A., Lopez Moreno, I.: Generalized end-to-end loss for speaker verification. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 4879–4883 (2018)
26. World Health Organization: World Report on Hearing. WHO, Geneva (2021)